

EXPERIMENT 2

Write a Python Program to Solve the 8-Queen Problem

Aim

To develop a Python program that solves the **8-Queen Problem** using the **Backtracking Algorithm** by placing eight queens on a chessboard such that no two queens attack each other.

Objective

- To understand the Backtracking Search Algorithm.
- To solve the 8-Queen Constraint Satisfaction Problem (CSP).
- To place eight queens safely on an 8×8 chessboard.
- To ensure that no two queens share the same row, column, or diagonal.

Theory

The **8-Queen Problem** is a well-known Artificial Intelligence problem in which eight queens must be placed on an 8×8 chessboard so that no queen attacks another queen.

A queen can attack another queen if they are in the:

- Same row
- Same column
- Same diagonal

The **Backtracking Algorithm** is used to solve this problem by placing one queen at a time. If a conflict occurs, the algorithm removes the previously placed queen and tries another position. This process continues until all eight queens are placed successfully.

Algorithm

Step 1: Initialize an empty 8×8 chessboard.

Step 2: Start placing queens from the first column.

Step 3: Check each row in the current column.

Step 4: Verify whether placing a queen is safe by checking:

- Left row

- Upper-left diagonal
- Lower-left diagonal

Step 5: If the position is safe:

- Place the queen.
- Move to the next column.

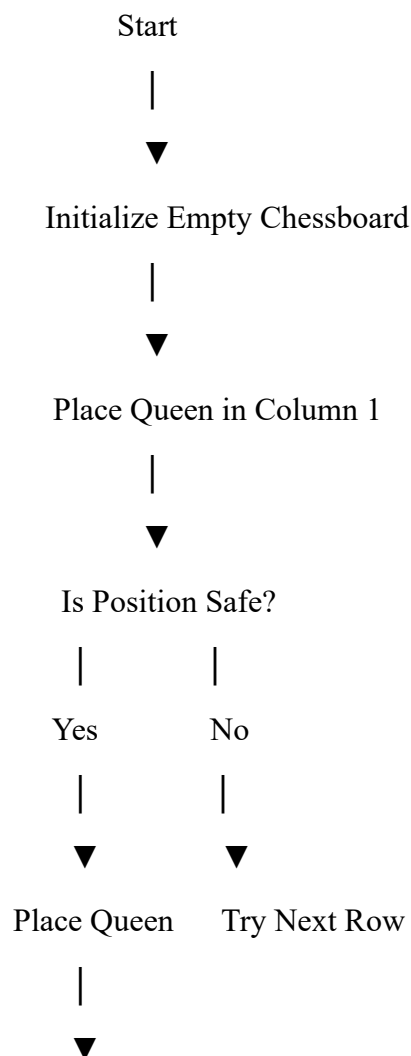
Step 6: If no safe position exists:

- Remove the previously placed queen.
- Try another row (Backtracking).

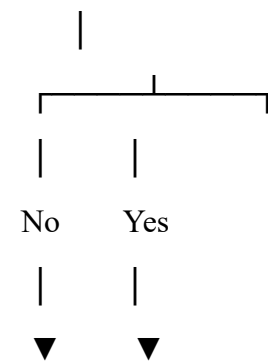
Step 7: Repeat until all eight queens are placed.

Step 8: Display the final chessboard.

Flowchart



Next Column Reached?



Continue Display Solution



Backtracking if Needed



Stop

Python Program

N = 8

```
def print_board(board):
```

```
    for row in board:
```

```
        print(" ".join(row))
```

```
def is_safe(board, row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
        if board[row][i] == "Q":
```

```
            return False
```

```
# Check upper diagonal
```

```
i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j] == "Q":
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
# Check lower diagonal
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j] == "Q":
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```

```
def solve(board, col):
```

```
    if col >= N:
```

```
        return True
```

```
    for row in range(N):
```

```
        if is_safe(board, row, col):
```

```
            board[row][col] = "Q"
```

```

        if solve(board, col + 1):
            return True

        board[row][col] = "."

    return False

board = [ "." for _ in range(N)] for _ in range(N)

if solve(board, 0):
    print("Solution Found:\n")
    print_board(board)
else:
    print("No Solution Exists")

```

Sample Output

Solution Found:

```

Q.....
....Q...
.....Q
.....Q..
..Q.....
.....Q.
.Q.....
...Q....

```

Explanation of Output

- Q represents a Queen.
- . represents an empty square.

- Every queen is placed so that no two queens are in the same row, column, or diagonal.

Result

The Python program successfully solved the **8-Queen Problem** using the **Backtracking Algorithm** and displayed a valid arrangement of eight queens on the chessboard without any conflicts.

Conclusion

The **8-Queen Problem** is a classical **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. The **Backtracking Algorithm** efficiently searches for a valid solution by exploring possible queen placements and backtracking whenever a conflict occurs. This experiment demonstrates how backtracking can solve complex combinatorial problems while satisfying all constraints, making it an important technique in AI and optimization.

```
EXP1.py X
C:\Users\Varshini M> OneDrive - Desktop -> Here!!!> ARTIFICIAL INTELLIGENCE> VSC EXPs> EXP1.py> ...

1  N = 8
2
3  def print_board(board):
4      for row in board:
5          print(" ".join(row))
6
7  def is_safe(board, row, col):
8
9      # Check left side
10     for i in range(col):
11         if board[row][i] == "Q":
12             return False
13
14     # Check upper diagonal
15     i, j = row, col
16     while i >= 0 and j >= 0:
17         if board[i][j] == "Q":
18             return False
19         i -= 1
20         j -= 1
21
22     # Check lower diagonal
23     i, j = row, col
24     while i < N and j >= 0:
25         if board[i][j] == "Q":
26             return False
27         i += 1
28         j -= 1
29
30     return True
31
32 def solve(board, col):
33
34     if col >= N:
35         return True
36
37     for row in range(N):
38
39         if is_safe(board, row, col):
40
41             board[row][col] = "Q"
42
43             if solve(board, col + 1):
44                 return True
45
46             board[row][col] = "."
47
48     return False
49
50 board = [ "." for _ in range(N) for _ in range(N) ]
51
52 if solve(board, 0):
53     print("Solution Found:\n")
54     print_board(board)
55 else:
56     print("No Solution Exists")
57
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Varshini M\AppData\Local\Pr
i M/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/EXP1.py"
Solution Found:

Q . . . . .
. . . . Q .
. . . Q . .
. . . . . Q
. Q . . . .
. . . Q . .
. . . . Q .
. . Q . . .

PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code>
```